

Resolución de problemas y algoritmos

Dra. Jessica Andrea Carballido

jessicarballido@gmail.com



Dpto. de Ciencias e Ingeniería de la Computación

UNIVERSIDAD NACIONAL DEL SUR



RECURSIVIDAD

recursion

Recursividad: los sí y los NO en los planteos



Sí	NO
Caso base: cuando es y resultado en ese caso.	HABLAR DE IMPLEMENTACION
Caso general: cuando es, como obtengo el resultado a partir del resultado para instancia menor, y quién es la instancia menor.	Llamo recursivamente...
Ponerle de título el problema que se quiere resolver <i>(suma de elementos de N, cantidad de dígitos de N, etc..)</i>	Asigno... Leo... Ejecuto...
Operaciones y expresiones matemáticas ($N < 10$; $N \bmod 10$...)	La función... El procedimiento...

Dígitos mellizos



Un número N se dice *ConDigMellizos* si posee al menos **un par de dígitos iguales adyacentes**. Dos dígitos son adyacentes si ocupan posiciones consecutivas en un número. Si tiene un dígito, no es *ConDigMellizos*.

Por ejemplo $N=7242$ no es *ConDigMellizos* en cambio $N=5331$ es *ConDigMellizos*

Problema: Escriba un planteo recursivo e implemente una función en Pascal para determinar si un número N cumple la propiedad *ConDigMellizos*.

N es ConDigMellizos?

Caso trivial 1: Un número N con un dígito no es *ConDigMellizos*.

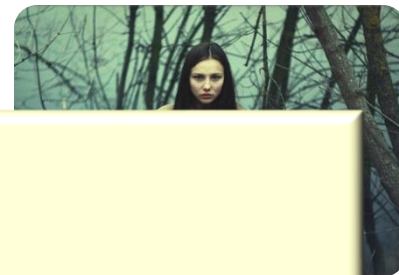
Caso trivial 2: Si N tiene mas de un digito y sus dos dígitos menos significativos son iguales es *ConDigMellizos*.

Caso Recursivo: Un número N que no verifica los casos triviales es *ConDigMellizos* si N' es *ConDigMellizos*, siendo N' igual a N sin su dígito menos significativo.

```
function ConDigMellizos(N: integer): boolean;  
  // función recursiva q dado un num. Natural determina si  
  // tiene dos dígitos adyacentes iguales  
  begin  
    if N < 10 then  
      ConDigMellizos:=false  
    else  
      if (N mod 10) = ((N div 10) mod 10)  
      then ConDigMellizos := true  
      else  
        ConDigMellizos:= ConDigMellizos (N div 10)  
      end;  
    end;
```

Es muy importante
documentar el código

Recursividad



```
Program prueba;  
Var numero: integer;  
function ConDigMellizos(N: integer): boolean;  
begin  
  if N < 10 then  
    ConDigMellizos:=false  
  else  
    if (N mod 10) = ((N div 10) mod 10)  
      then    ConDigMellizos := true  
      else  
        ConDigMellizos:= ConDigMellizos (N div 10)  
    end;  
  Begin  
    Writeln('Ingrese un numero'); readln(numero);  
    If (ConDigMellizos(numero) = TRUE) then  
      writeln('El numero ', numero, ' es con dígitos mellizos')  
    else  
      writeln('El numero ', numero, ' es con dígitos mellizos')  
  End.
```


Recursividad

```
Program prueba;  
Var numero: integer;
```

```
Begin  
  Writeln('Ingrese un numero');  
  readln(numero);  
  If (ConDigMellizos(numero) = TRUE) then  
    writeln('El numero ', numero, ' es con  
    dígitos mellizos')  
  else  
    writeln('El numero ', numero, ' es con  
    dígitos mellizos')  
End.
```



numero	4551

El usuario ingresa el número 4551

```
writeln(ingrese un numero );
readln(numero);
If (ConDigMellizos(numero) = TRUE) tl
    writeln('El numero ', numero
```

Resuelve el problema para N=4551

```
function ConDigMellizos(N: integer): boolean;
begin
    if N < 10 then
        ConDigMellizos:=false
    else
        if (N mod 10) = ((N div 10) mod 10)
        then
            ConDigMellizos := true
        else
            ConDigMellizos:= ConDigMellizos (N div 10)
    end;
```

Llamo para que se resuelva 455



numero	4551
N	4551
ConDigMellizos	

Recursividad

Resuelve el problema para N=455

```
function ConDigMellizos(N: integer): boolean;  
begin  
  if N < 10 then  
    ConDigMellizos:=false  
  else  
    if (N mod 10) = ((N div 10) mod 10)  
    then  
      ConDigMellizos := true  
    else  
      ConDigMellizos:= ConDigMellizos (N div 10)  
    end;  
end;
```



numero	4551
N	4551
ConDigMellizos	
N	455
ConDigMellizos	

```

Begin
Writeln('Ingrese un numero');
readln(numero);
If (ConDigMellizos(numero) = TRUE) then
    writeln('El numero ', numero, ' es con
    dígitos mellizos')
else
    writeln('El numero ', numero, ' es con
    dígitos mellizos')
End.

```

Resuelve el problema para N=4551

```

function ConDigMellizos(N: integer): boolean;
begin
    if N < 10 then
        ConDigMellizos:=false
    else
        if (N mod 10) = ((N div 10) mod 10)
        then
            ConDigMellizos := true
        else
            ConDigMellizos:= ConDigMellizos (N div 10)
        end;

```

Llamo para que se re

numero	4551
--------	------

N	4551
---	------

ConDigMellizos	
-----------------------	--

N	455
---	-----

ConDigMellizos	
-----------------------	--

Resuelve el problema para N=455

```

function ConDigMellizos(N: integer): boolean;
begin
    if N < 10 then
        ConDigMellizos:=false
    else
        if (N mod 10) = ((N div 10) mod 10)
        then
            ConDigMellizos := true
        else
            ConDigMellizos:= ConDigMellizos (N div 10)
        end;

```

Recursividad

Resuelve el problema para N=455

```
function ConDigMellizos(N: integer): boolean;  
begin  
  if N < 10 then  
    ConDigMellizos := FALSE  
  else  
    if (N mod 10) = ((N div 10) mod 10)  
    then  
      ConDigMellizos := TRUE  
    else  
      ConDigMellizos:= ConDigMellizos (N div 10)  
    end;  
end;
```



numero	4551
N	4551
ConDigMellizos	
N	455
ConDigMellizos	TRUE

Resuelve el problema para N=4551

```
function ConDigMellizos(N: integer): boolean;  
begin  
  if N < 10 then  
    ConDigMellizos := FALSE  
  else  
    if (N mod 10) = ((N div 10) mod 10)  
    then  
      ConDigMellizos := TRUE  
    else  
      ConDigMellizos:= ConDigMellizos (N div 10)  
    end;  
  end;
```

Resuelve el problema para N=455

```
function ConDigMellizos(N: integer): boolean;  
begin  
  if N < 10 then  
    ConDigMellizos := FALSE  
  else  
    if (N mod 10) = ((N div 10) mod 10)  
    then  
      ConDigMellizos := TRUE  
    else  
      ConDigMellizos:= ConDigMellizos (N div 10)  
    end;
```



numero	4551
N	4551
ConDigMellizos	
N	455
ConDigMellizos	TRUE

Resuelve el problema para N=4551

```
function ConDigMellizos(N: integer): boolean;  
begin  
  if N < 10 then  
    ConDigMellizos:=FALSE  
  else  
    if (N mod 10) = ((N div 10) mod 10)  
    then  
      ConDigMellizos := TRUE  
    else  
      ConDigMellizos:= TRUE  
    end;  
end;
```



numero	4551
N	4551
<i>ConDigMellizos</i>	

Resuelve el problema para N=4551

```
function ConDigMellizos(N: integer): boolean;  
begin  
  if N < 10 then  
    ConDigMellizos := FALSE  
  else  
    if (N mod 10) = ((N div 10) mod 10)  
    then  
      ConDigMellizos := TRUE  
    else  
      ConDigMellizos := TRUE  
    end;  
end;
```



numero	4551
N	4551
<i>ConDigMellizos</i>	TRUE

```
Begin
Writeln('Ingrese un numero');
readln(numero);
If (ConDigMellizos(numero) = TRUE) then
    writeln('El numero ', numero, ' es con
dígitos mellizos')
else
    writeln('El numero ', numero, ' es con
dígitos mellizos')
End.
```

```
function ConDigMellizos(N: integer): boolean;
begin
  if N < 10 then
    ConDigMellizos := FALSE
  else
    if (N mod 10) = ((N div 10) mod 10)
    then
      ConDigMellizos := TRUE
    else
      ConDigMellizos:= TRUE
    end;
  end;
```



numero	4551
N	4551
ConDigMellizos	TRUE

Recursividad

```
Begin
  Writeln('Ingrese un numero');
  readln(numero);
  If (TRUE = TRUE) then
    writeln('El numero ', numero, ' es con
    dígitos mellizos')
  else
    writeln('El numero ', numero, ' es con
    dígitos mellizos')
End.
```

FIN



Recursividad

```
Begin
Writeln('Ingresa un numero');
readln(numero);
If (ConDigMellizos(numero) = TRUE) then
    writeln('El numero ', numero, ' es con
    dígitos mellizos')
else
    writeln('El numero ', numero, ' es con
    dígitos mellizos')
End.
```



numero	29

El usuario ingresa el número 29

Recursividad

Resuelve el problema para N=29

```
function ConDigMellizos(N: integer): boolean;  
begin  
  if N < 10 then  
    ConDigMellizos:=false  
  else  
    if (N mod 10) = ((N div 10) mod 10)  
    then  
      ConDigMellizos := true  
    else  
      ConDigMellizos:= ConDigMellizos (N div 10)  
    end;  
end;
```

Llamo para que se resuelva 2



numero	29
N	29
ConDigMellizos	

Recursividad

Resuelve el problema para N=2

```
function ConDigMellizos(N: integer): boolean;  
begin  
  if N < 10 then  
    ConDigMellizos:=false  
  else  
    if (N mod 10) = ((N div 10) mod 10)  
    then  
      ConDigMellizos := true  
    else  
      ConDigMellizos:= ConDigMellizos (N div 10)  
    end;  
end;
```



numero	29
N	29
ConDigMellizos	
N	2
ConDigMellizos	

Recursividad

Resuelve el problema para N=2

```
function ConDigMellizos(N: integer): boolean;  
begin  
  if N < 10 then  
    ConDigMellizos:=false  
  else  
    if (N mod 10) = ((N div 10) mod 10)  
    then  
      ConDigMellizos := true  
    else  
      ConDigMellizos:= ConDigMellizos (N div 10)  
    end;  
end;
```



numero	29
N	29
ConDigMellizos	
N	2
ConDigMellizos	FALSE

Resuelve el problema para N=29

```
function ConDigMellizos(N: integer): boolean;  
begin  
  if N < 10 then  
    ConDigMellizos:=false  
  else  
    if (N mod 10) = ((N div 10) mod 10)  
    then  
      ConDigMellizos := true  
    else  
      ConDigMellizos:= ConDigMellizos (N div 10)  
    end;  
end;
```

Resuelve el problema para N=2

```
function ConDigMellizos(N: integer): boolean;  
begin  
  if N < 10 then  
    ConDigMellizos:=false  
  else  
    if (N mod 10) = ((N div 10) mod 10)  
    then  
      ConDigMellizos := true  
    else  
      ConDigMellizos:= ConDigMellizos (N div 10)  
    end;
```



numero	29
N	29
ConDigMellizos	
N	2
ConDigMellizos	FALSE

Resuelve el problema para N=29

```
function ConDigMellizos(N: integer): boolean;  
begin  
  if N < 10 then  
    ConDigMellizos:=false  
  else  
    if (N mod 10) = ((N div 10) mod 10)  
    then  
      ConDigMellizos := true  
    else  
      ConDigMellizos:= FALSE  
    end;  
end;
```



numero	29
N	29
<i>ConDigMellizos</i>	

Resuelve el problema para N=29

```
function ConDigMellizos(N: integer): boolean;  
begin  
  if N < 10 then  
    ConDigMellizos:=false  
  else  
    if (N mod 10) = ((N div 10) mod 10)  
    then  
      ConDigMellizos := true  
    else  
      ConDigMellizos:= FALSE  
    end;  
end;
```



numero	29
N	29
<i>ConDigMellizos</i>	FALSE


```
Begin
Writeln('Ingrese un numero');
readln(numero);
If (ConDigMellizos(numero) = TRUE) then
    writeln('El numero ', numero, ' es con
dígitos mellizos')
else
    writeln('El numero ', numero, ' es con
dígitos mellizos')
End.
```

```
function ConDigMellizos(N: integer): boolean;
begin
    if N < 10 then
        ConDigMellizos:=false
    else
        if (N mod 10) = ((N div 10) mod 10)
        then
            ConDigMellizos := true
        else
            ConDigMellizos:= FALSE
        end;
    end;
```



numero	29
N	29
ConDigMellizos	FALSE

Recursividad

```
Begin
Writeln('Ingresa un numero');
readln(numero);
If (FALSE = TRUE) then
    writeln('El numero ', numero, ' es con
    dígitos mellizos')
else
    writeln('El numero ', numero, ' es con
    dígitos mellizos')
End.
```

FIN



Recursividad



Escriba programas completos para resolver los siguientes problemas (usando como primitiva la función `conDigitosMellizos`).

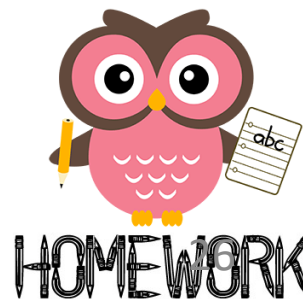
Podés armar también un único programa con un menú de 3 opciones (¡¡que sean procedimientos!!) para que el usuario elija cual de los problemas quiere resolver.

Problema 1: Dada una secuencia de números leída de teclado (finaliza con 0) se desea contar **cuántos** números cumplen la propiedad `ConDigMellizos`.

Problema 2: Dada una secuencia de números leída de teclado (finaliza con 0) se desea **determinar si todos** cumplen con la propiedad `ConDigMellizos`.

Problema 3: Dada una secuencia de números leída de teclado (finaliza con 0) se desea **determinar si al menos dos elementos adyacentes** cumplen con la propiedad `ConDigMellizos`.

¡Repetir suponiendo que conocemos con antelación la longitud!

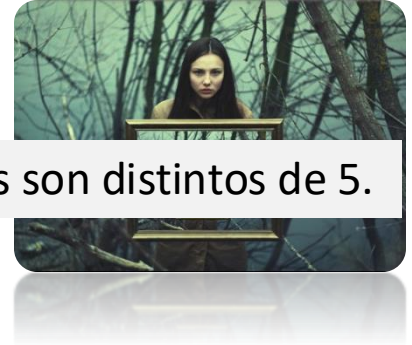




Recordamos las prácticas de buena programación

- ✓ Usar nombres significativos en las variables
- ✓ Comentar el código con explicaciones claras en las partes que se considere necesario (documentar).
 - ✓ Modularizar correctamente
 - ✓ Indentar las instrucciones prolijamente

Recursividad



Problema: Dado un número natural N , se desea saber si todos sus dígitos son distintos de 5.

Planteo “sin5(N)”

CASO TRIVIAL 1: Si N tiene un dígito, y es 5, sin5(N) es FALSO.

CASO TRIVIAL 2: Si N tiene un dígito, y es distinto de 5, sin5(N) es VERDADERO.

CASO TRIVIAL 3: Si N tiene más de un dígito y el último dígito de N es 5, sin5(N) es FALSO.

CASO GENERAL: Si N tiene más de un dígito y el último dígito de N es distinto de 5, sin5(N) es igual a sin5(N').

Donde N' es N sin su último dígito.

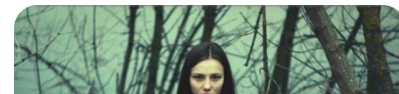
```
function sin5(N: integer): boolean;  
Begin  
  if (N < 9)  
    then  
      if (N = 5) then      sin5 := FALSE  
        else            sin5 := TRUE  
    else  
      if (N mod 10 = 5) then sin5 := FALSE  
        else sin5 := sin5 (N div 10);  
End;
```

Recursividad



```
function sin5(N: integer): boolean;  
Begin  
  if (N < 9)  
    then  
      sin5 := (N <> 5)  
    else  
      if (N mod 10 = 5) then sin5 := FALSE  
      else sin5 := sin5 (N div 10);  
End;
```

Recursividad



Problema: Dado un número natural N , se reemplazar cada dígito di tal que $di=5$ por 1.

Planteo “por1(N)”

CASO TRIVIAL 1: Si N tiene un dígito, y es 5, $\text{por1}(N)$ es 1.

CASO TRIVIAL 2: Si N tiene un dígito, y es distinto de 5, $\text{por1}(N)$ es N .



CASO GENERAL 1: Si N tiene más de un dígito y el último dígito de N es igual a 5, $\text{por1}(N)$ es $\text{por1}(N')$ seguido de 1.

CASO GENERAL 2: Si N tiene más de un dígito y el último dígito de N es distinto de 5, $\text{por1}(N)$ es $\text{por1}(N')$ seguido del último dígito de N .

Donde N' es N sin su último dígito.

```
function por1(N: integer): integer;  
Begin  
  if (N<9)  
    then  
      if (N=5) then      por1 := 1  
        else          por1 := N  
    else  
      if (N mod 10 = 5) then      por1 := por1(N div 10) * 10 + 1  
        else          por1 := por1(N div 10) * 10 + (N mod 10);  
End;
```

Recursividad

Problemas

Dados dos números naturales $N1$ y $N2$ mayores que cero, se desea:

a. Determinar si $N1$ es **prefijo** de $N2$.

Ej.: Prefijo(12, 123) = Sí

Prefijo(12, 12) = Sí

Prefijo(12, 132) = No.

Prefijo (12, 9) = No

b. Determinar si $N1$ es **sufijo** de $N2$.

Ej.: Sufijo(12, 312) = Sí

Sufijo(12, 12) = Sí

Sufijo(12, 132) = No



Problemas

Dados dos números naturales $N1$ y $N2$, se desea:

a. Determinar si $N1$ es prefijo de $N2$.

Planteo $N1$ es prefijo de $N2$

CB1: Si $N1=N2$, $N1$ es prefijo de $N2$.

CB2: Si $N1>N2$, $N1$ no es prefijo de $N2$.

CG: Si $N2>N1$, $N1$ es prefijo de $N2$ si $N1$ es prefijo de $N2$ sin su último dígito.

Problemas

Dados dos números naturales N1 y N2, se desea:

a. Determinar si N1 es **prefijo** de N2.

```
function prefijo(N1, N2: integer): boolean;  
{ Esta función asume q se ingresan dos números positivos, y retorna verdadero si el  
primero es prefijo del segundo }  
begin  
  if (N1=N2) then prefijo:=true // caso trivial  
    else  
      if (N1>N2)  
        then  
          prefijo:=false // caso trivial  
        else  
          prefijo:=prefijo(N1, N2 div 10); // caso general  
      end;  
    end;  
end;
```



Es muy importante
documentar el código

Problemas

Dados dos números naturales $N1$ y $N2$, se desea:

b. Determinar si $N1$ es **sufijo** de $N2$.

Planteo $N1$ es sufijo de $N2$

CB1: Si $N1=0$, $N1$ es sufijo de $N2$.

CB2: Si el último dígito de $N1$ es distinto del último dígito de $N2$, $N1$ NO ES sufijo de $N2$.

CG: Si el último dígito de $N1$ es igual al último dígito de $N2$, $N1$ es sufijo de $N2$ si $N1'$ es sufijo de $N2'$.

Donde **$N1'$ y $N2'$** son $N1$ y $N2$ sin sus últimos dígitos respectivamente.

Problemas

Dados dos números naturales N1 y N2, se desea:

b. Determinar si N1 es **sufijo** de N2.

```
function sufijo(N1, N2: integer): boolean;  
begin  
  if (N1=0) then sufijo:=true  
    else  
      if (N1 mod 10)<>(N2 mod 10)  
        then sufijo:=false  
        else  
          sufijo:=sufijo(N1 div 10, N2 div 10);  
        end;  
    end;  
end;
```

Problemas

Dados dos números naturales N y M mayores que cero, se desea:

c. Calcular la **suma de productos de los dígitos** de la siguiente manera (ASUMIR que ambos números tienen igual cantidad de dígitos):

$$\text{Si } N = n_k \dots n_2 n_1 \text{ y } M = m_k \dots m_2 m_1$$

$$\text{La Suma de Productos de } (N, M) = n_1 * m_1 + n_2 * m_2 + \dots + n_k * m_k$$

Planteo: Suma de Productos (SP) de los dígitos de N y M

CB: Si N y M tienen un dígito, la SP de los dígitos de N y M es $N * M$.

CG: Si N y M tienen más de un dígito, la SP de los dígitos de N y M es la suma de productos de N' y M' + el producto de los últimos dígitos de N y M.

Donde N' y M' son N y M sin sus últimos dígitos respectivamente.

Recursividad



```
26 function sumaPRODS(n, m: integer): integer;  
27 begin  
28   if (n<10) and (m<10)  
29   then  
30     sumaPRODS := n * m  
31   else  
32     sumaPRODS := (n mod 10)*(m mod 10)+sumaPRODS(n div 10, m div 10);  
33   end;
```

Ejercicio 1: Escribir un programa para que, dada una secuencia de PARES de números, se muestre mostrar por pantalla la suma de productos de los dígitos de cada par de números usando esta función como primitiva.

Puede asumir que antes de leer la secuencia tiene como dato de entrada la longitud de la secuencia.

```
Ingrese la cantidad de pares que tendrá la secuencia: 3  
Ingrese ahora la secuencia:  
23 51 6 2 231 502  
Para 23 y 51: 13  
Para 6 y 2: 12  
Para 231 y 502: 12
```

ASUMIMOS QUE CADA PAR DE NÚMEROS TIENE IGUAL CANTIDAD DE DIGITOS.

Recursividad



```
1 program Ultimos;
2 var n1, n2, i, longitud:integer;
3
4 function sumaPRODS(n, m: integer): integer;
5 begin
6   if (n<10) and (m<10)
7   then
8     sumaPRODS := n * m
9   else
10    sumaPRODS := (n mod 10)*(m mod 10)+sumaPRODS(n div 10, m div 10);
11 end;
12 begin
13   write(' Ingrese la cantidad de pares que tendrá la secuencia: ');
14   readln(longitud);
15   writeln(' Ingrese ahora la secuencia: ');
16   write(' ');
17   for i:=1 to longitud do
18   begin
19     read(n1, n2);
20     writeln(' Para ', n1, ' y ', n2, ': ', sumaPRODS(n1, n2));
21   end;
22 end.
```

Recursividad



```
4 function sumaPRODS (n, m:integer):integer;  
5 begin if (n < 10)  
6 and (m < 10) then sumaPRODS:= n * m  
7 else  
8 sumaPRODS:= (n mod 10) * (m mod 10) + sumaPRODS (n div 10, m div 10);  
9 end;
```

¿Y si quiero mostrar la máxima suma de productos,
y cuáles son los números q la consiguieron?
Suponiendo 0 0 como terminadores ;)

```
Ingrese una secuencia de pares de números con igual cantidad de dígitos.  
Finalice con 0 0  
12 42 99 88 2341 1111 0 0  
La suma de productos para 12 y 42 es 8  
La suma de productos para 99 y 88 es 144  
La suma de productos para 2341 y 1111 es 10  
  
La maxima suma de productos fue 144 obtenida con los números 99 y 88
```



```

1  program LAST;
2  var a, b, aMax, bMax, sum, sumMax: integer;
3
4  function sumaPRODS (n, m:integer):integer;
5  begin
6  if (n < 10) and (m < 10)
7  then
8      sumaPRODS:= n * m
9  else
10     sumaPRODS:= (n mod 10) * (m mod 10) + sumaPRODS (n div 10, m div 10);
11 end;
12
13 begin
14 sumMax := 0;
15 writeln('Ingrese una secuencia de pares de números con igual cantidad de dígitos. ');
16 writeln('Finalice con 0 0');
17 repeat
18     read(a, b);
19     if (a <> 0) and (b <> 0)
20     then
21     begin
22         sum := sumaPRODS(a, b);
23         writeln('La suma de productos para ', a, ' y ', b, ' es ', sum);
24         if (sum > sumMax)
25         then
26         begin
27             sumMax := sum;
28             aMax := a;
29             bMax := b;
30         end;
31     end
32 until (a = 0) and (b = 0);
33 writeln;
34 writeln('La maxima suma de productos fue ', sumMax, ' obtenida con los números ', aMax, ' y ', bMax);
35 end

```

Recursividad



```
26 function sumaPRODS(n, m: integer): integer;  
27 begin  
28   if (n<10) and (m<10)  
29   then  
30     sumaPRODS := n * m  
31   else  
32     sumaPRODS := (n mod 10)*(m mod 10)+sumaPRODS(n div 10, m div 10);  
33   end;
```

Ejercicio 2: Escribir un programa para que, dada una secuencia de PARES de números se determine si PARA ALGUNO de los PARES de números naturales, la suma de producto da un múltiplo de 5, usando esta función como primitiva.

El final de la secuencia se representa con el par 0 0.

Ingresa la secuencia finalizando con 0 0:

7 8 66 72 251 314 5 1 0 0

Al menos uno de los pares tiene "suma de producto" que es múltiplo de 5.
El primer par de números encontrado que cumple es: 251 314

ASUMIMOS QUE CADA PAR DE NÚMEROS TIENE IGUAL CANTIDAD DE DIGITOS.

Recursividad

```
1 program Ultimos;
2 var n1, n2:integer;
3     hay: boolean;
4 function sumaPRODS(n, m: integer): integer;
5 begin
6   if (n<10) and (m<10)
7   then
8     sumaPRODS := n * m
9   else
10    sumaPRODS := (n mod 10)*(m mod 10)+sumaPRODS(n div 10, m div 10);
11  end;
12 begin
13  writeln(' Ingrese la secuencia finalizando con 0 0: ');
14  write(' ');
15  hay := false;
16 repeat
17   read(n1, n2);
18   if (n1<>0) and (n2<>0)
19   then
20     if (sumaPRODS(n1, n2) MOD 5 = 0) then hay:=TRUE;
21 until ((n1=0) and (n2=0)) OR (hay);
22 if (hay)
23 then
24 begin
25   writeln('Al menos uno de los pares tiene "suma de producto" que es multiplo de 5.');
```

26 writeln('El primer par de números encontrado que cumple es: ', n1, ' ', n2);

27 end

28 else

29 writeln('Ningún par cumple la condición.')

30 end.

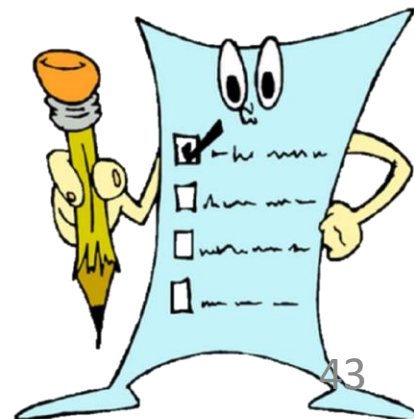




ENCUESTAS OFICIALES

Tu opinión nos importa!
Y nos sirve para mejorar.

Asegurate de completarlas en el SIU Guaraní.



Fechas

Para pedir fecha

Para coordinar la consulta



examen

FINAL

**SOLO SI APROBAS LOS PARCIALES (o recu)
Y NO RENDIS COLOQUIO**

Teoría

Defina, describa, compare, justifique, ilustre con un ejemplo

Práctica



MUCHAS GRACIAS!!!

Mucha suerte,
y hasta pronto...

